MAKÜ
BURDUR MEHMET AKİF ERSOY ÜNİVERSİTESİ

VERİ YAPILARI VE ALGORİTMALAR

HAFTA 3
ASİMPTOTİK NOTASYONLAR

Dr. Öğr. Üyesi Ebru AYDOĞAN

Bucak Bilgisayar ve Bilişim Fakültesi

Ders İçeriği: Hafta 3

- Karmaşıklık Analizi
- Asimptotik Notasyonlar
- Karmaşıklık Analizi – Hesaplama
- Karmaşıklık Analizi – Özyineleme
- Örnek Karmaşıklık Analizi
- Sıkça Yapılan Hatalar

Karmaşıklık Analizi

- En Küçük Sayıyı Bulma T(n) Hesabı

	Birim Zaman	Tekrar Sayısı	Toplam En Kötü	Toplam Ortalama	Toplam En İyi
float bulEnKucuk(float A[]) {	-	-	-		
float enkucuk;	-	-	-		
int k ;	-	-	-		
enkucuk=A[0];	1	1	1	1	1
for(k=1;k<n;k++)	1,1,1	1, n, (n-1)	2n	2n	2n
if (A[k]<enkucuk)	1	n-1	n-1	n-1	n-1
enkucuk=A[k];	1	n-1	n-1	(n-1)/2	0
return enkucuk;	1	1	1	1	1
}	-	-	-		
			T(n) = 4n	(7n+1)/2	3n+1

- Dikkat edilirse enkucuk=A[k] satırında en kötü durum alınmıştır. En iyi durum 0 iken ortalama durum (n-1)/2 olarak hesaplanabilir.

Karmaşıklık Analizi

- İki matrisin toplanması için T(n) hesabı

	Birim Zaman	Tekrar Sayısı	Toplam
void toplaMatris (A,B,C) {	-	-	-
int A[n][m], B[n][m], C[n][m];	-	-	-
int i,j ;	-	-	-
for(i=0;i<n;i++)	1,1,1	1,(n+1),n	2n+2
for(j=0;j<m;j++)	1,1,1	n(1,(m+1),m)	n(2m+2)
C[i][j]=A[i][j]+B[i][j];	1	nm	nm
}	-	-	-
			T(n,m)=3nm+4n+2

Karmaşıklık Analizi

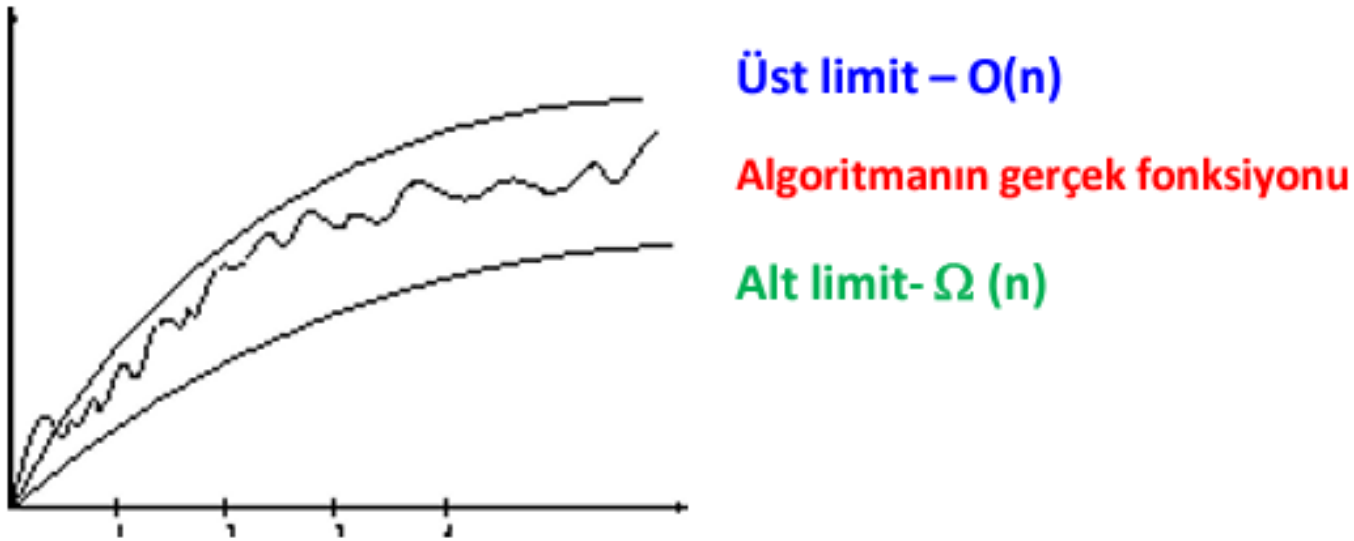
- n yerine satır, m yerine sütun konulursa ;
  - T(satır, sütun) = 3* satır * sütun + 4 * satır + 2 , bağımsız parametreler matrisin satır ve sütun değerleridir.
- Matrisler kare matris olarak alınırsa n=m ise;
  - T(n)= $3n^2 + 4n + 2$ şeklinde bir ikinci derece denkleme karşı gelir.
- Burada
  - T(n)= $3n^2 + 4n + 2$ gibi bir ifade üzerinde en önemli parametre hangi bölümdür?
  - Başka bir deyişle yürütme hızını hangi kısım çok etkiler?

Karmaşıklık Analizi

- Karmaşıklık (Complexity); bir algoritmanın çok sayıda parametre karşısındaki değişimini gösteren ifadelerdir.
- Çalışma (Yürütme) zamanını daha doğru bir şekilde bulmak için kullanılırlar.
- Bir algoritma için yürütme zamanı, maliyet veya karmaşıklık hesaplarında üç duruma bakılır:
  - En iyi durum: algoritmada her şeyin istenildiği gibi olduğundaki durumdur.
  - En kötü durum: algoritmada oluşabilecek en kötü senaryo göz önüne alınır.
  - Ortalama durum ise oluşabilecek en kötü ve en iyi durumun ortalamasıdır.

Karmaşıklık Analizi

- Genel strateji bir algoritmanın alt ve üst limitlerinin bulunmasıdır.



- Diğer bir deyişle bir algoritmanın en iyi durum(best case) ve en kötü durum(worst case) yürütme zamanının analizi yapılır.

Karmaşıklık Analizi

- Bir algoritmanın performansı iç ve dış faktörlere bağlıdır.
  - İç
    - Algoritma verimliliği
    - Çalıştırmak için gereken zaman
    - Çalıştırmak için gereken yer (bellek alanı)
  - Dış
    - Girdi verisinin büyüklüğü
    - Bilgisayarın hızı
    - Derleyicinin kalitesi
- Karmaşıklık iç faktörlerle ve daha çok da zamanla ilgilidir.

Karmaşıklık Analizi

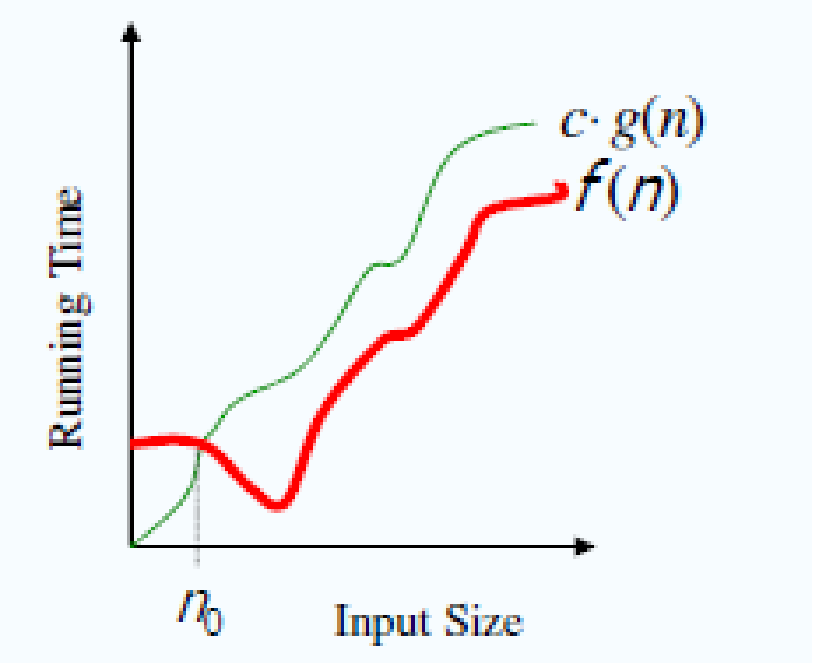
- Genel olarak, az sayıdaki parametreler için karmaşıklıkla ilgilenilmez;
  - eleman sayısı n'nin sonsuza gitmesi durumunda algoritmanın maliyet hesabının davranışını görmek
  - diğer benzer işleri yapan algoritmalarla karşılaştırmak için kullanılır.
- Karmaşıklığı ifade edebilmek için asimptotik ifadeler kullanılmaktadır.
- Bu amaçla O(n) (O notasyonu), Ω(n) (Omega notasyonu), θ(n) (Teta notasyonu) gibi tanımlamalara baş vurulur.

Asimptotik Notasyonlar – Big-O

- Büyük-Oh (Big-O) Notasyonu bir algoritmanın büyüme hızını gösterir.
- Büyüme hızı ise bir algoritmanın performansını yansıtan en iyi göstergedir.
- Big-O, bir algoritmanın performansını en iyi tanımlayan matematiksel formüldür ve algoritmanın iç detaylarına bakılarak elde edilir.
- Big O hesaplanırken, yürütme zamanı bağıntısındaki en büyük artışı yöneten terim alınır ve diğerleri göz ardı edilir. Çünkü girdi boyutu sonsuza giderken büyüme hızını yönetecek olan, en büyük değişkenli terimdir.
- Big-O notasyonu, asimptotik üst sınır veya en kötü durum analizi olarak adlandırılır.

Asimptotik Notasyonlar – Big-O

- Bir algoritmanın çalışma süresi,
  - f(n)=O(g(n)) için;
  - f(n) ≤ c.g(n) ve n ≥ n_0 koşullarını sağlayan c *(sabit katsayı)* ve n_0 *(başlangıç noktası)* değerleri varsa f(n) ≤ c.g(n) ifadesi doğrudur.
  - g(n), f(n)'in asimptotik üst limiti olarak adlandırılır.
  - f(n)=O(g(n))'dir.



Asimptotik Notasyonlar – Big-O

- Örnek: $3n^2 + 2n + 5 \leq O(n^2)$ olduğunu gösteriniz.

  - $f(n) = 3n^2 + 2n + 5$ ve $g(n) = n^2$
  - $f(n) \leq c.g(n)$, $\quad n \geq n_0$

  - $3n^2 + 2n + 5 \leq c.n^2$
  - $2n + 5 \leq (c - 3).n^2$
  - $c = 4$ seçelim.
  - $2n + 5 \leq (4 - 3).n^2$ yani $2n + 5 \leq n^2$
  - $n \geq 3$ olur.

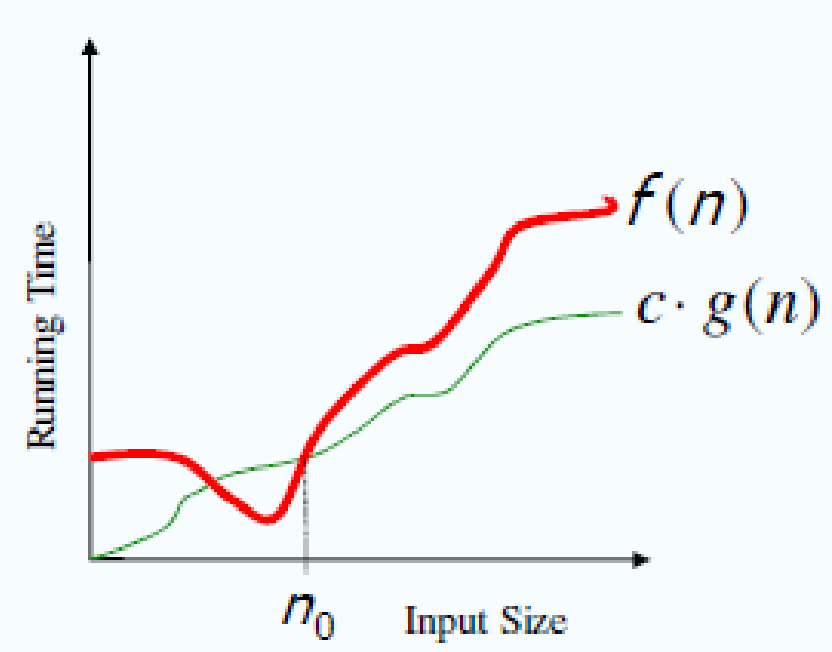
  - c=4 ve n_0=3 için
  - $3n^2 + 2n + 5 \leq 4.n^2$, n ≥ 3 eşitsizliği sağlanmış olur.

Asimptotik Notasyonlar – Big-O

- Big-O girdi verisinin büyüklüğünü gösteren n parametresine dayanan bir fonksiyondur.
- Örneğin n değerine bağlı olarak performansı (sabit a, b, c değerleri için) $an^2 + bn + c$ olan bir algoritmanın performansı O(n^2)'dir.
- n değeri arttıkça n^2 terimi baskın olacağı için Big-O notasyonunda sadece baskın olan terim kullanılır.(Üssü en büyük olan kullanılır.)
- Örnekler:
  - T(n) = 3n + 4 -> O(n)
  - T(n) = 4n -> O(n)
  - T(n,m) = 3nm + 4n + 2 -> 3nn +4n -> O(n^2)
  - T(n) = 3 n^3 + 2n + 4 -> O(n^3)

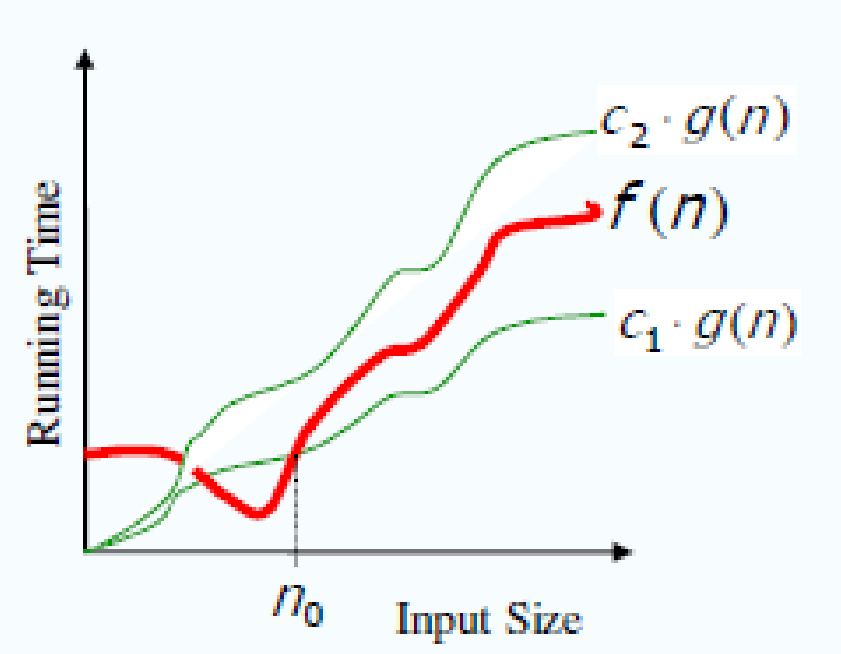
Asimptotik Notasyonlar – Big-Ω

- Asimptotik alt sınırı veya en iyi (best-case) çalışma süresini tanımlar.
- Bir algoritmanın çalışma süresi,
  - f(n)= Ω(g(n)) için;
  - c.g(n) ≤ f(n) ve n ≥ n_0 koşullarını sağlayan c ve n_0 değerleri varsa c.g(n) ≤ f(n) ifadesi doğrudur.



Asimptotik Notasyonlar – Big-Θ

- Asimptotik alt-üst arasındaki ortalama (avarage-case) çalışma süresini tanımlar.
- Bir algoritmanın çalışma süresi,
  - f(n)= Θ(g(n)) için;
  - c_1.g(n) ≤ f(n) ≤ c_2.g(n) ve n ≥ n_0 koşullarını sağlayan c_1, c_2 ve n_0 değerleri varsa c_1.g(n) ≤ f(n) ≤ c_2.g(n) ifadesi doğrudur.



Karmaşıklık Analizi – Hesaplama

- Big-O nasıl hesaplanır?

Kural 1: Döngüler

- Bir döngünün en fazla çalışma zamanı, döngü içindeki deyimlerin çalışma zamanının tekrar (iterasyon) sayısıyla çarpılması kadardır.

n defa çalışır

```
for (i=1; i<=n; i++)
{
    m = m + 2;
}
```

← Sabit zaman (c)

Toplam zaman = c * n = cn = O(N)

Karmaşıklık Analizi – Hesaplama

- Big-O nasıl hesaplanır?

Kural 2: İç İçe Döngüler

- Kural 1'e göre en içteki işlemin analizi yapılır. Toplam zaman ise bütün döngülerin çalışma sayılarının çarpımına eşittir.

Dış döngü *n* defa çalışır

```
for (i=1; i<=n; i++) {
   for (j=1; j<=n; j++) {
      k = k+1;
   }
}
```

Sabit zaman (c)

iç döngü *n* defa çalışır

Toplam zaman = c * n * n = cn^2 = $O(N^2)$

Karmaşıklık Analizi – Hesaplama

- Big-O nasıl hesaplanır?

Kural 3: Ardışık Deyimler

- Toplam zaman ise bütün deyimlerin çalışma zamanlarının toplamına eşittir.

Sabit zaman →

```
x = x +1;
for (i=1; i<=n; i++) {
    m = m + 2;
}
```

Sabit zaman →

n defa çalışır

Dış döngü n defa çalışır

```
for (i=1; i<=n; i++)
{
    for (j=1; j<=n; j++) {
        k = k+1;
    }
}
```

← Sabit zaman

iç döngü n defa çalışır

Toplam zaman = c0 + c1.n + c2. n^2 = O(N^2)

Karmaşıklık Analizi – Hesaplama

- Big-O nasıl hesaplanır?

Kural 4: if-else Deyimleri

- En kötü çalışma zamanı için if veya else kısmındaki yürütme zamanının hangisi büyükse o kısım toplama eklenir.

Sabit zaman →

```
if (i != n){
  return false;
}
```

} Sabit

Sabit zaman →

```
else
{
   for (i=1; i<=n; i++) {
      k = k+1;
   }
}
```

} else: Sabit * n defa çalışır

Toplam zaman = c0 + c1.n = O(N)

Karmaşıklık Analizi – Hesaplama

- Big-O nasıl hesaplanır?

Kural 5: Logaritmik Karmaşıklık

- Problemin büyüklüğünü belli oranda (genelde ½) azaltmak için sabit bir zaman harcanıyorsa bu algoritma O(log N) büyüme hızında artandır.
- Örnek: (binary search – İkili Arama):
  - N sayfalı bir sözlükten bir sözcük arama için
  - Sözlüğün orta kısmına bakılır
  - Sözcük ortaya göre sağda mı solda mı kaldığı bulunur
  - Bu işlem sağ veya solda sözcük bulunana kadar tekrarlanır.

Karmaşıklık Analizi – Hesaplama

```
int binary search(A,key,N)
    low=0, high=N-1 ————————→ O(1)
    while(low≤high)
        mid=(low+high)/2 ————→ O(1)
        if(A[mid]<key) ————→ O(1)
            low=mid+1
        if(A[mid]>key) ————→ O(1)
            high=mid-1;
        if(A[mid]=key) ————→ O(1)
            return mid
    Return not found ————→ O(1)
```

O(1)

O(logN)

O(logN)

- Her bir iterasyondan sonra, arama yapılacak eleman sayısı logaritmik olarak azalmaktadır. Çalışma süresi O(logN)'dir.

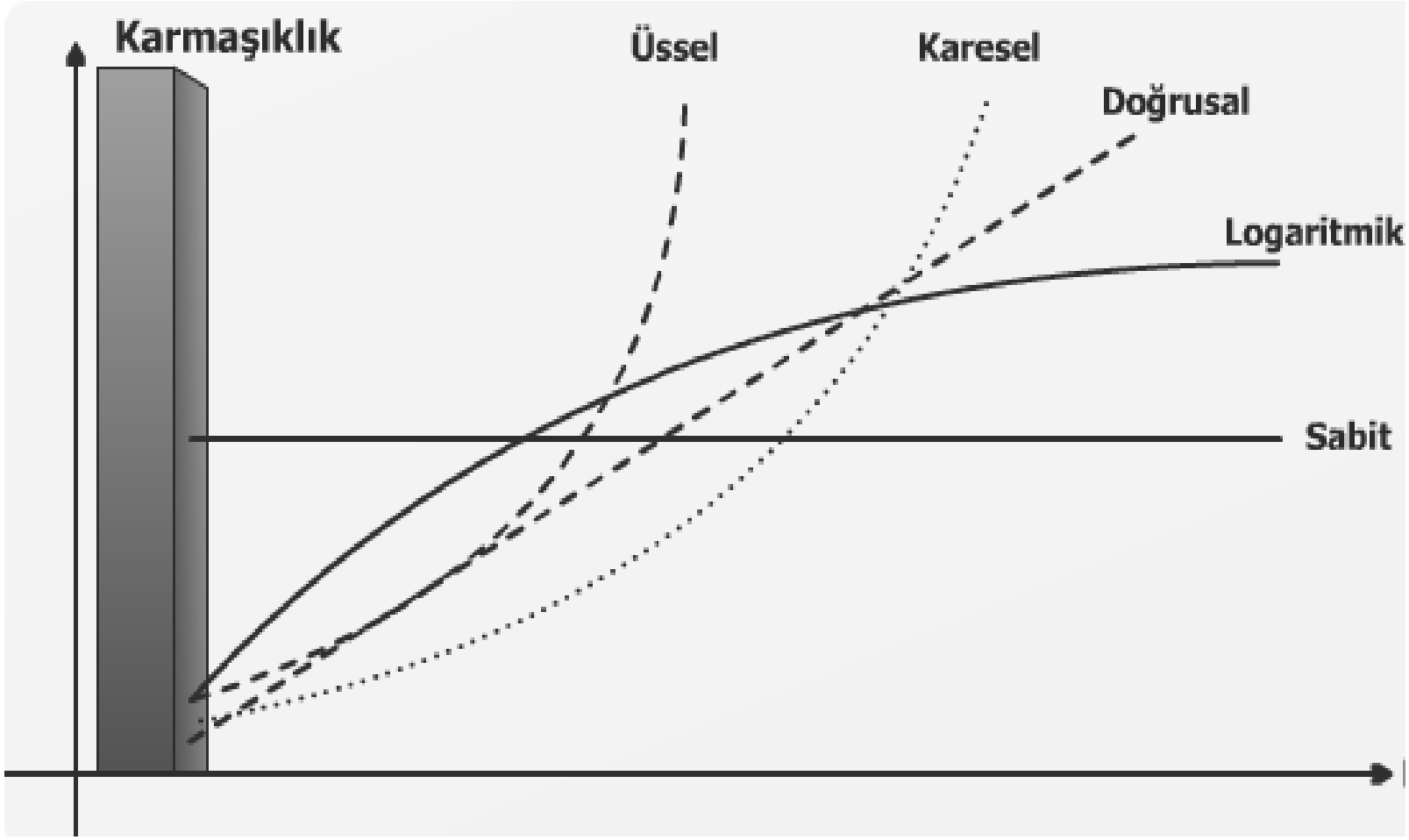
Karmaşıklık Analizi – Hesaplama

➢ Karşılaşılan Genel Fonksiyonlar

Zaman karmaşıklığı	**Örnek**
O(1) *sabit*	Bağlı listeye ilk eleman olarak ekleme yapma
O(log *N*) *log*	Sıralı bir dizide bir eleman arama
O(*N*) *lineer*	Sıralı olmayan bir dizide bir eleman arama
O(*N* log *N*) *n-log-n*	N elemanı böl-parçala-yut yöntemiyle sıralama
O(N^2) *ikinci dereceden*	Bir grafikte iki düğüm arasındaki en kısa yolu bulma
O(N^3) *üçüncü dereceden*	Ardarda gerçekleştirilen lineer denklemler
O(2^N) *üssel*	Hanoi'nin Kuleleri problemi

Karmaşıklık Analizi – Hesaplama

- Karşılaşılan Genel Fonksiyonlar



Karmaşıklık Analizi – Özyineleme

➢ Özyinelemeli Yapılar

- Özyineleme (rekürsif) kavramı matematikte olduğu gibi programlamada da önemli bir yere sahiptir.
- Bu tekniğin, günümüzde de birçok problemin çözümünde etkin olarak kullanıldığını söyleyebiliriz.
- Özyineleme yöntemi arama ve sıralama algoritmalarında, veri yapılarında ve iteratif yöntemle çözülen birçok problemin çözümünde alternatif olarak kullanılmaktadır.

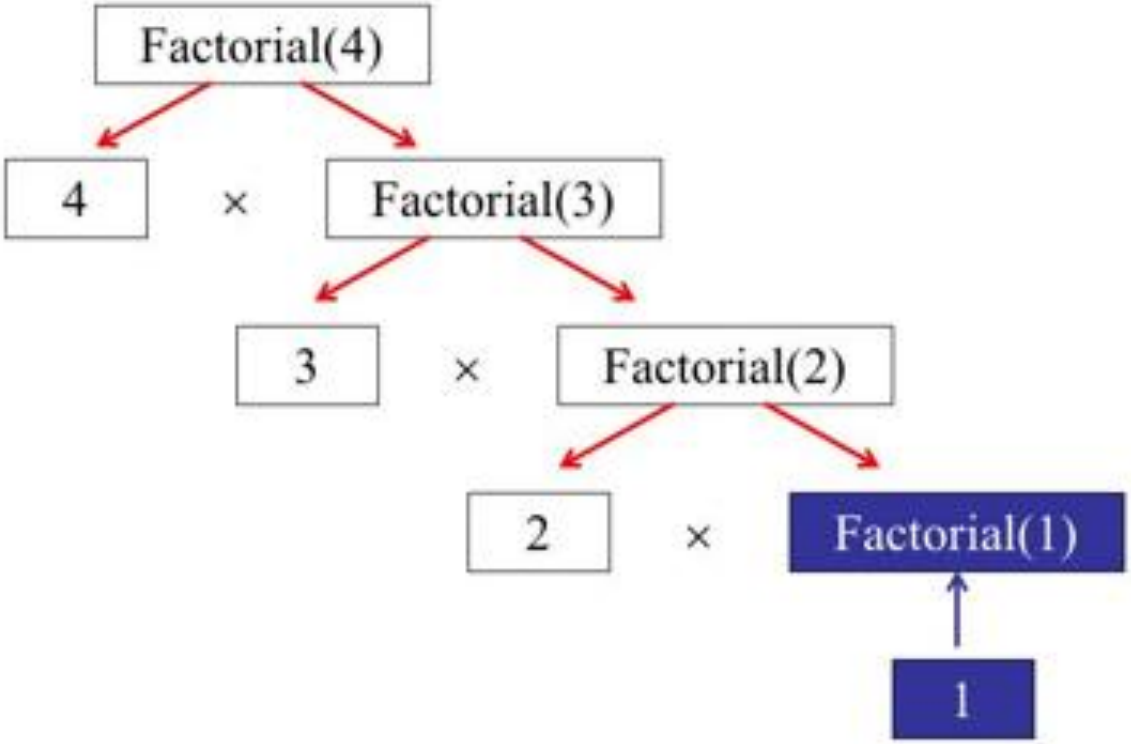
Karmaşıklık Analizi – Özyineleme

➢ Özyinelemeli Yapılar

- Çoğunlukla, iteratif olarak çözülebilen problemler, özyinelemeli fonksiyonlarla çok daha kısa kod yazarak çözülebilir.
- Bilgisayar bilimi açısından özyineleme tanımı, bir fonksiyonun görevini yapabilmesi için kendi kendini çağırması şeklinde yapılmaktadır.
- Başka bir ifadeyle özyineleme, belirli bir koşul karşılanana kadar kendisini, bir veya daha fazla kez çağıran işlev veya algoritmayı kullanan bir programlama tekniğidir.

Karmaşıklık Analizi – Özyineleme

- Özyinelemeli Yapılar
  - Örnek: Faktöriyel İşlemi



Karmaşıklık Analizi – Özyineleme

➢ Özyinelemeli Kodların Analizi

- Örnek: Aşağıdaki faktöriyel hesaplama kodlardan hangisi daha iyi?

For döngüsü kullanarak faktoriyel hesaplama:

```
1  private int Faktoriyel(int sayi)
2  {
3      int fakt = 1;
4      for (int i = 1; i <= sayi; i++)
5      {
6          fakt = fakt * i;
7      }
8      return fakt;
9  }
```

Rekürsif fonksiyon kullanarak faktoriyel hesaplama:

```
1  private int Faktoriyel(intv sayi)
2  {
3      if(sayi == 0) return 1; else return sayi * Faktoriyel(sayi - 1);
4  }
```

Karmaşıklık Analizi – Özyineleme

- Özyinelemeli Kodların Analizi

```
int factorial1(int n)
  {  if (n<=1)
              return 1;
  else {
    fact = 1;                    O(1)
    for (k=2;k<=n;k++)
        fact *= k;               O(n)
    return fact;                 O(1)
  }
}
```

For döngüsü ile Faktöriyel: O(N) çalışma zamanı

Karmaşıklık Analizi – Özyineleme

- Özyinelemeli Kodların Analizi

```
int factorial (int n) {
if (n<=1)                          n
    return 1;                      1
else
return (n * factorial(n-1));       n-1
}
```

Rekürsif Faktöriyel:
T(N)= n+1+(n-1) = 2n ise;
T(N) = O(N) çalışma zamanı

Karmaşıklık Analizi – Özyineleme

➢ Özyinelemeli Kodların Analizi

- Zaman karmaşıklığı açısından rekürsif kod ile döngü arasında fark yoktur.
- Ancak bellek karmaşıklığı açısından arada fark vardır.
- Rekürsif durumda N tane durum öncelikle belleğe yüklenir (O(N)) ve sonra hesaplanır. (Stack yapısı şeklinde çalışır. İleri de anlatılacak)
- Ancak döngüde her döngü sonunda işlem hesaplanır ve böylece belleğe maliyeti sadece O(1)'dir.
- Rekürsif kodlar için bellek gereksinimi fazladır.
- Veri için bellek gereksinimi, doğrudan program içerisinde kullanılan değişken/sabit sayısı ve türüne, veri yapısına bağlıdır.

Sıkça Yapılan Hatalar

- Karmaşıklığı bulmak için sadece döngüleri saymakla yetinmeyin.
  - 2 içi içe döngünün 1'den N^2 'ye kadar döndüğünü düşünürsek karmaşıklık $O(N^4)$ olur.
- $O(2N^2)$ veya $O(N^2+N)$ gibi ifadeler kullanmayın.
  - Sadece baskın terim kullanılır.
  - Öndeki sabitler kaldırılır.
- İç içe döngüler karmaşıklığı direk etkilerken art arda gelen döngüler karmaşıklığı etkilemez.